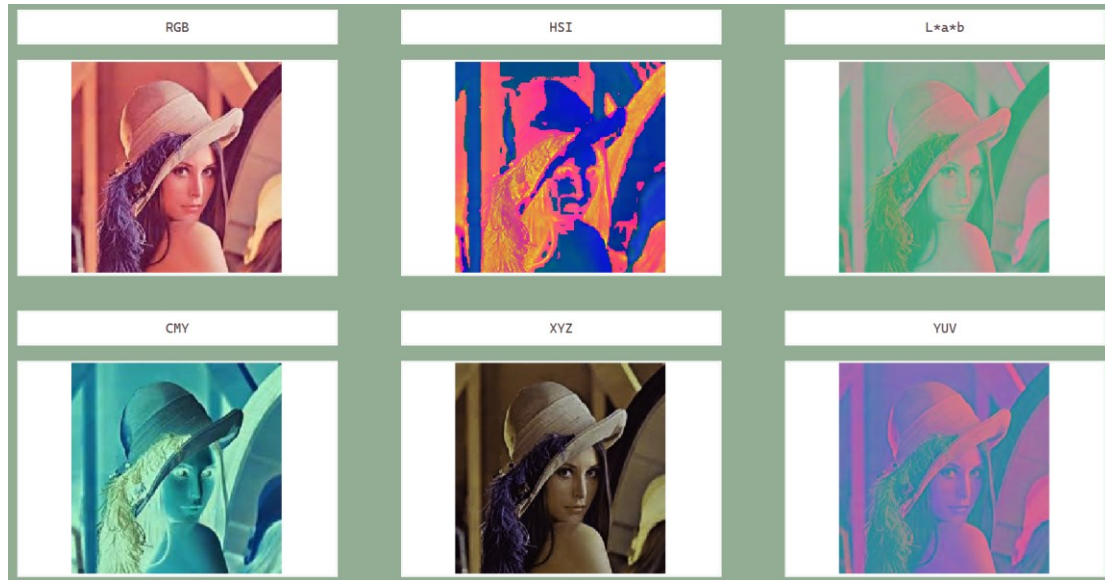


Digital Image Processing

Homework 5

PART1: Design a program for converting image in RGB, CMY, HSI, XYZ, L*a*b*, YUV color planes.



Color Model Conversion Algorithms:

1. RGB to CMY

Algorithm: The CMY model is inverted from RGB components

- $C = 255 - R$
- $M = 255 - G$
- $Y = 255 - B$
-

2. RGB to HSI

Algorithm:

- **Hue (H):** The calculation involves determining the angle between color and axis using trigonometric functions, and hue angle within the range $[0, 360]$ degrees
- **Saturation (S):** The ratio of the lightness of the pixel to the maximum possible value
- **Intensity (I):** The average of the R, G, and B components

The conversion helps separate chromatic information (H and S) from intensity, help to analyze visual aspects like shading.

3. XYZ to Lab

Algorithm:

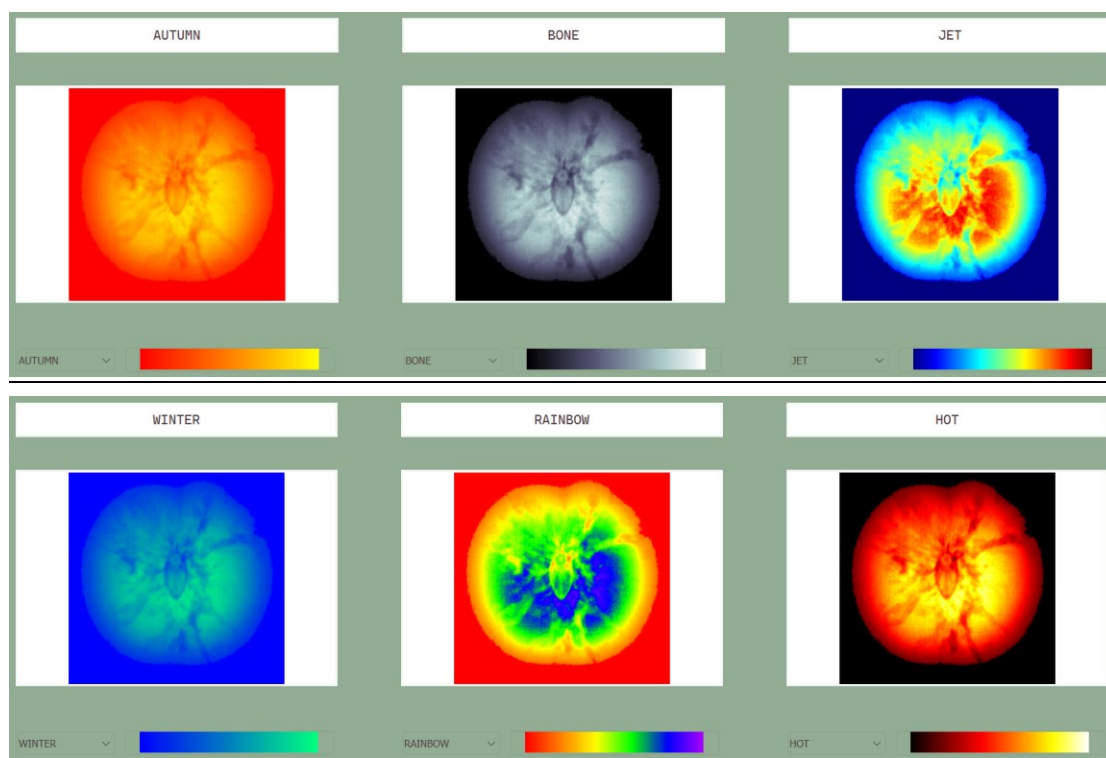
The conversion create a perceptually uniform color space, normalizing the color perception and emphasizes color contrast, which is helpful in segmentation and analysis.

4. *RGB to YUV*

Algorithm:

- **Y (Luminance):** Compute using a weighted sum of RGB components
 - **U and V** capture the chromatic information, providing color contrast data
- The YUV space is valuable for applications where separation of brightness and color is needed, like video compression.

PART2: Develop a program to display the grayscale and pseudo-color images for comparison and show the color table (color bar) with its corresponding grayscale.



Pseudo-color Algorithm

Algorithm:

OpenCV's `applyColorMap()` function converts grayscale images into pseudo-color representations. The program also generates a color bar showing the mapping between grayscale values and the colors used.

PART3: Design a program for image segmentation by color clustering using the k-means algorithm. Compare the color segmentation results using RGB, HSI, and L*a*b* color planes (using $k = 2$).

RGB

K=2



K=10



K=50

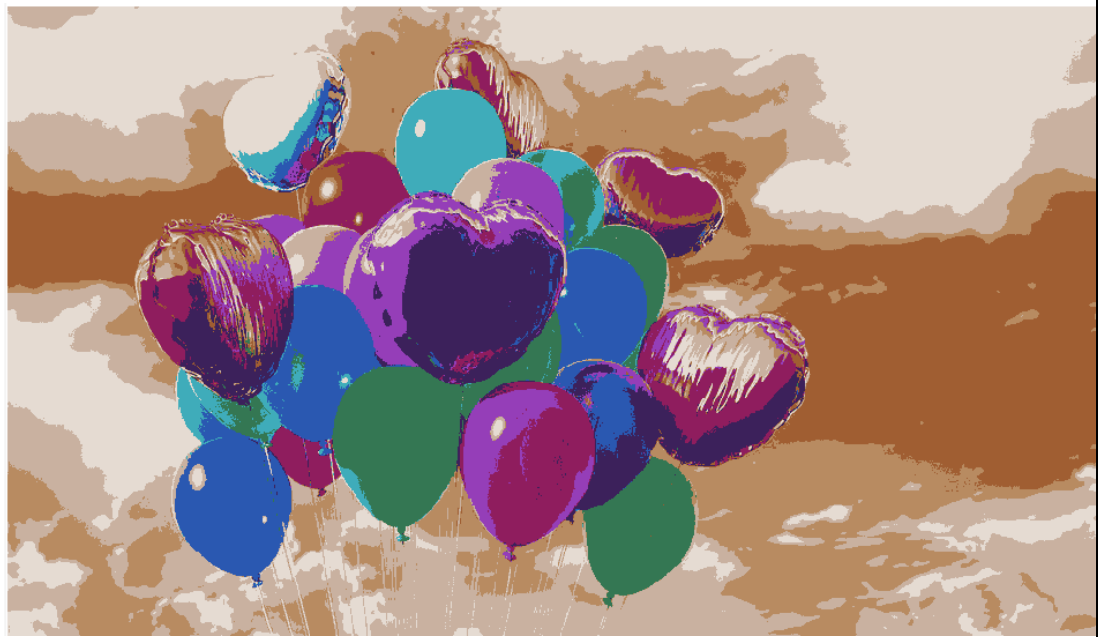


L*a*b

K=2



K=10



K=50

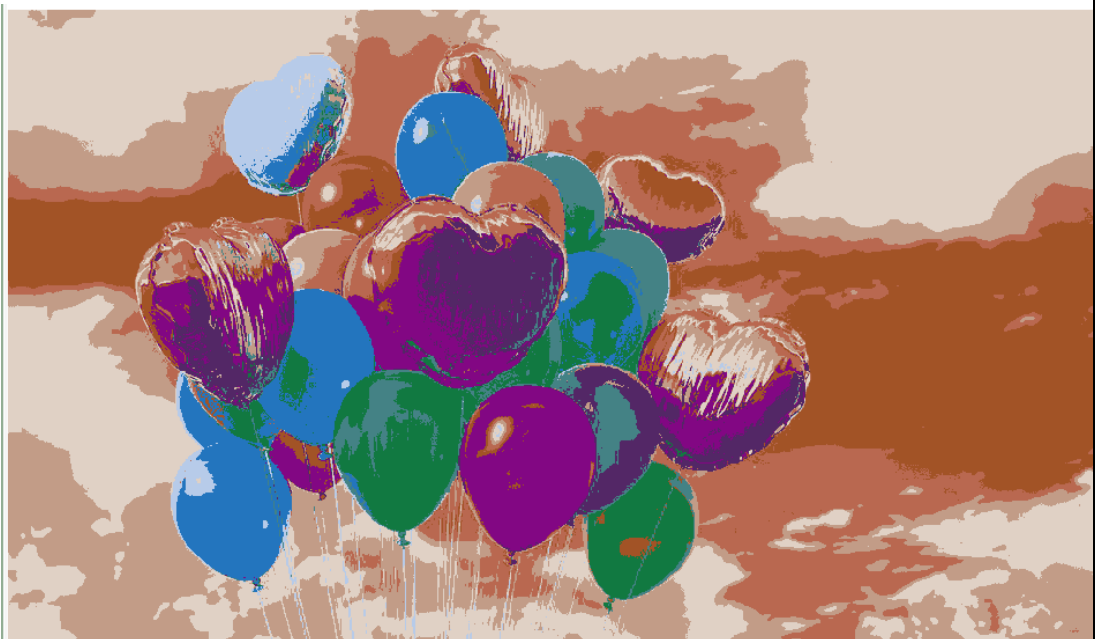


HSI

K=2



K=10



K=50



K-means Clustering Algorithm

K-means is an unsupervised learning algorithm used for clustering. Each pixel in the image is treated as a data point in a color space, and k-means groups similar pixels into k clusters.

The algorithm minimizes the sum of squared distances between pixels and the cluster centers, effectively grouping similar colors.

Algorithm Steps:

Step 1: Data Preparation

The program first converts the input image into the selected color space (RGB, HSI, or *Lab**).

Step 2: Normalization

The color components are normalized to fit in the range [0, 1]. OpenCV `kmeans()` function is used for the clustering. The function takes the reshaped image matrix and splits the image pixels into k clusters.

Step 3: Assigning Colors:

The program assigns distinct colors to each cluster, creating a segmented version of the image. Each cluster is colored differently, allowing the user to visually differentiate the segments. In RGB, HSI, and *Lab**, distinct cluster colors are assigned by mapping each pixel to the nearest cluster center.

Step 4: Visualization:

The segmented image is converted back to RGB color space to be displayed.